Milestone One - Vector Runtime Analysis

Pseudocode Statement	Cost	Times Executed	Total
OPEN file	1	1	1
IF file does not open	1	1	1
CREATE temporary vector	1	1	1
WHILE lines remain	1	n + 1	n + 1
READ next line	1	n	n
SPLIT line by commas	1	n	n
IF fewer than 2 pieces	1	n	n
CREATE new Course	1	n	n
SAVE course number	1	n	n
SAVE course title	1	n	n
IF more than 2 pieces	1	n	n
SAVE prerequisites	1	≤ n	≤ n
ADD Course to vector	1*	n	n
CLOSE file	1	1	1
FOR each course	1	n + 1	n + 1
FOR each prerequisite	1	≤ n(p + 1)	≤ n(p + 1)
SEARCH vector for prerequisite	n	≤ np	$\leq n^2 p$
IF prerequisite is not found	1	≤ np	≤ np
REPLACE courses vector	1*	1	1
RETURN true	1	1	1

Big-O result: $O(n^2 p)$

I counted ADD and REPLACE as constant time operations in order to keep the calculations simple. I also used n for the number of courses and p for the maximum number of prerequisites each course can have. If the number of prerequisites stays small, the runtime would be $O(n^2)$.

Vector Evaluation

I think the vector was the easiest of the data structures to work with since it keeps all the courses together in one single list. Adding new courses was pretty straightforward, and I can access them by their position within the vector. The downside is that searching for a specific

course might take longer when more courses are added since it might have to check every course before finding the right one. My pseudocode also has to make separate copies of the vector to sort out the courses before printing them in order.

For memory, the vector uses O9N) space to store courses. It can have extra space reserved for more courses, and my code creates a temporary vector while loading the file. That uses more memory, but it lets me check out course information before replacing the original vector.

Milestone Two – Hash Table Runtime Analysis

Pseudocode Statement	Cost	Times Executed	Total
OPEN file	1	1	1
IF file does not open	1	1	1
CREATE temporary course list	1	1	1
WHILE another line exists	1	n + 1	n + 1
READ line	1	n	n
SPLIT line by commas	1	n	n
IF fewer than 2 items	1	n	n
CREATE new Course	1	n	n
SAVE course number	1	n	n
SAVE course title	1	n	n
IF more than 2 items	1	n	n
SAVE prerequisites	1	≤ n	≤ n
ADD Course to temporary list	1*	n	n
CLOSE file	1	1	1
FOR each course	1	n + 1	n + 1
FOR each prerequisite	1	≤ n(p + 1)	≤ n(p + 1)
SEARCH temporary list for prerequisite	n	≤ np	$\leq n^2 p$
IF prerequisite not found	1	≤ np	≤ np
CLEAR hash table	1*	1	1
FOR each Course to insert	1	n +1	n + 1
INSERT Course into hash table	O(n)	n	$O(n^2)$
RETURN true	1	1	1

Big-O result: $O(n^2+n^2)$ **or** $O(n^2 p)$

I used n for the number of courses and p for the max number of prerequisites each course can have. My pseudocode checked prerequisites by using a temporary list before adding any courses to the hash table. That way, it still had to search through the list. With a fixed maximum number of prerequisites, the worst-case runtime would be $O(n^2)$.

Hash Table Evaluation

The hash table would be useful for finding specific courses since it uses the course number as a key instead of searching through courses like the vector does. This makes looking up course information a lot faster. One of the downsides is that the hash tables don't automatically keep everything in order, so my pseudocode had to copy the courses into a separate list and sort them before printing results. There's also the possibility of collisions when different course numbers end up at the same location in the table, which can slow it down.

For memory, the hash table uses O(n) space to store courses, but it needs extra space for its buckets. My code uses a temporary list to check out the course info before adding it to the hash table and another list when sorting out courses. It makes it so it needs more memory compared to just simply storing the courses themselves.

Milestone Three – Binary Search Tree Runtime Analysis

Pseudocode Statement	Cost	Times Executed	Total
OPEN file	1	1	1
IF file does not open	1	1	1
CREATE courseList	1	1	1
WHILE lines remain	1	n + 1	n + 1
READ next line	1	n	n
SPLIT line by commas	1	n	n
IF fewer than 2 pieces	1	n	n
CREATE new Course	1	n	n
SAVE course number	1	n	n
SAVE course title	1	n	n
IF more than 2 pieces	1	n	n
SAVE prerequisites	1	≤ n	≤ n
ADD Course to courseList	1*	n	n
CLOSE file	1	1	1
FOR each Course	1	n + 1	n + 1
FOR each prerequisite	1	≤ n(p + 1)	≤ n(p + 1)
SEARCH courseList for prerequisite	n	≤ np	$\leq n^2 p$
IF prerequisite not found	1	≤ np	≤ np
CREATE temporary tree	1	1	1
FOR each Course to insert	1	n + 1	n + 1
CALL InsertCourse	O(n)	n	$O(n^2)$
REPLACE courses tree	1*	1	1
RETURN true	1	1	1

Big-O results: $O(n^2 p + n^2)$

Like the earlier charts, I used n for the number of courses and p for the max number of prerequisites each course can have. My pseudocode checks the prerequisites by using a course list before adding anything to the binary search tree. Inserting courses can take longer if the tree

becomes unbalanced since it might have to use several nodes to find where each course belongs. With a fixed number of prerequisites, the worst-case runtime would be $O(n^2)$

Binary Search Tree Evaluation

The binary search tree seems to have the better advantage over the other two since it can keep the courses organized by their numbers as they're added. I wouldn't need to sort anything separately before printing the course list like i did with the vector and hash table. My pseudocode uses an in-order traversal to print the courses in alphanumeric order by going through the left nodes, printing the current course, then going to the right nodes. Searching can be quicker when the tree is more balanced. The downside is that if the tree becomes unbalanced, searching and inserting courses can take longer since it may not have to go through several nodes to find the right one.

For memory, the binary search tree uses O(n) space to store the courses, but each node also needs pointers to its left and right children. My code creates a temporary course list and tree while also loading the file, which still uses more memory. The recursive functions also need extra memory to keep track of where they are in the tree, especially if the tree has a lot of levels to it.

I recommend using the binary search tree for the course advising program. The program needs to display the courses in alphanumeric order, and I wouldn't have to sort them separately like I did with the vector or hash table. I can just use an in-order traversal to print them in the correct order. Searching for a course can also be faster since it doesn't have to go through every course to find the one it's looking for. The only thing I'd have to be careful about is the tree becoming unbalanced. If that happens, searching for courses or adding new ones could take

longer. Even with that to consider, I'd still choose the binary search tree since it makes more sense for what the program needs to do.